FOCAFIRE — Divisão de Tarefas do Projecto

Equipa: Argon2 | Desafio: NASA Space Apps 2026 — Harmonization of MODIS and VIIRS Hot Spots

Foco: Angola | Versão: 1.0 | Data: 17-10-2026 | Dono: M1 (Coordenação)

Capacidade: 4–5 membros x 1h/dia = 28–35h agregadas no Sprint 1

1. Objectivo deste documento

Garantir que cada membro sabe exactamente o que fazer do início ao fim do projecto,

sem sobreposição e sem bloqueios. Nenhum membro faz o trabalho do outro.

O M1 coordena e desbloqueia, não centraliza o código.

2. Equipa e papéis

ID	Função	Responsabilidade principal	Não faz
M1	Coordenação + GitHub	Board, backlog, reviews, demo, docs transversais	Pipeline, API ou mapa completos
M2	Python / Dados	FIRMS, CSV, normalização, metodologia	Frontend ou API Node
M3	Backend + Base de dados	Express, schema SQL, API de leitura	Lógica científica, mapa
M4	Frontend	Next.js, Leaflet, Recharts, filtros	Pipeline, schema SQL
M5	UI/UX + Testes + Docs	Layout, checklist manual, apresentação	Lógica de dados ou API

Se forem 4 membros: M5 é absorvido por M1 (docs/demo) e M4 (layout/teste visual).

3. Stack e ferramentas por membro

Cada membro instala e usa apenas o que está na sua linha. Ferramenta partilhada: GitHub + VS Code.

ID	Linguagens	Frameworks / Bibliotecas	Ferramentas / Serviços	Instala na semana 1
M1	Markdown	—	GitHub (Projects, PRs, branch protection), Git CLI, WhatsApp/Discord para standup	Git
M2	Python 3.11+	Pandas, pytest	FIRMS API/Downloads, CSV/JSON, `venv`	Python + `pip install pandas pytest`
M3	TypeScript (Node 20+)	Express	PostgreSQL/PostGIS (Sprint 2; fallback JSON/SQLite), Vitest + Supertest, npm	Node.js + npm
M4	TypeScript (React)	Next.js 14+, Leaflet + react-leaflet, Recharts	npm, `npm run dev`/`build`	Node.js (mesma do M3)

ID	Linguagens	Frameworks / Bibliotecas	Ferramentas / Serviços	Instala na semana 1
M5	—	Figma ou wireframe simples	Markdown, screenshots, vídeo 90s	Nenhum extra

3.1 Notas de stack (do plano do projecto)

• **M2 é o único dono de Python.** Ninguém escreve `pipeline` em TypeScript nem Python no `frontend//backend/`. Se M3 precisar de dados, consome a API ou o JSON gerado por M2.

• **M3 escreve o backend em Node/Express/TypeScript.** Não duplica a lógica científica (validação de regras) — isso é de M2. M3 só lê, valida parâmetros e pagina.

• **M4 escreve só em Next.js/React/TypeScript.** Não lê CSV nem liga directamente à DB — só consome `/api/*`.

• **PostgreSQL + PostGIS entram no Sprint 2 (M3).** No Sprint 1, M3 trabalha com `detections_mock.json` lido directamente pelo Express. O schema SQL é escrito no Sprint 1 mas não exige servidor ativo.

• **Leaflet é obrigatório, Recharts depende do tempo.** Se F08/F09 atrasarem, M4 entrega mapa + filtros primeiro e o gráfico fica para Sprint 2.

• **Ferramentas de teste por dono:** `pytest` só em `data-pipeline/tests/`, `Vitest` só em `backend/tests/`, frontend testado manualmente com checklist em F12.

• **CI (GitHub Actions) — M1 cria, M2/M3 só escrevem os comandos:** job Python = `pip install -r data-pipeline/requirements.txt && pytest`, job Node = `npm ci && npm test && npm run build`. Sem linters pesados na semana 1.

3.2 Proibições por papéis

Nunca fazer	Quem
Commitar `.env` ou MAP_KEY	Todos
Escrever código no módulo de outro membro	Todos
Push directo em `main`	Todos
Adicionar dependência nova sem avisar o M1	M2-M5
Instalar Postgres se F09/estiver mock-ok	M3 (Sprint 1)

4. Princípios de trabalho

1. 1 issue = 1 entrega verificável com dono único.
2. 1 branch por issue, vida < 24h. Sem push directo na `main`.
3. Todo PR exige 1 review. Merge via Merge Commit ou Squash, depois Delete Branch.
4. Nunca commitar `.env`, MAP_KEY, CSV/JSON grande. Só `data/samples/` pequeno.
5. Frontend e Backend nascem com MOCK. Troca por dados reais só na integração.
6. Distinguir sempre `REAL FIRMS` vs `AMOSTRA/TESTE` na UI e nos docs.
7. Falar em `detecções/observações`, nunca em `incêndios` como contagem infalível.
8. Bloqueio documentado também é entrega. Usar label `blocked`.

Definição de Pronto (DoR): issue tem dono, descrição, critérios de aceitação e mock/contrato disponível.

Definição de Feito (DoD): código na `main` + teste passa + docs actualizados + sem segredo + demo local funciona.

5. Mapa de dependências

```
F04 (FIRMS) -> F05 (amostras) -> F06 (pipeline) -> integração real
\ \ \
\ \-> F08 (schema) -> F09 (API) -> F11 (mapa real)
\-> F10 (layout mock) -> F11 (mapa mock) -> F12 (demo mock)
F02 (workflow) + F03 (âmbito) -> desbloqueiam todos
F07 (metodologia) acompanha F05/F06
```

Regra: ninguém espera pela FIRMS para codar. Usa-se mock até F05 fechar.

6. Sprint 1 — Fundação e validação (17–23 Out 2026)

Objectivo: repo organizado, acesso FIRMS verificado, pipeline inicial, mapa mock, demo mínima.

6.1 M1 — Coordenação

ID	Tarefa	Entrega	Aceitação	Est.
F01	Criar repo GitHub	Repo + `.gitignore` + `.env.example` + `README` + protecção `main` + convites	Push directo bloqueado, repo acessível	1h
F02	Fluxo de trabalho	`docs/workflow.md` + PR template	Qualquer membro abre branch->PR sem perguntar	0.5h
F03	Âmbito MVP	`docs/decisions.md` (bbox Angola, período 2024-01-01 a 2024-12-31, PT+EN, DB adiada, IN/OUT)	Decisão assinada, sem pressupostos	0.5h
F12a	Board e rituais	Board Todo/In Progress/Done, standup async ontem/hoje/bloqueio	Board reflecte realidade diária	0.5h/dia
HYG	Higiene	`docs/known_issues.md`, `data/samples/README.md`, `docs/FIRMS_ACCESS.md` moldes	M2 só preenche	1h

6.2 M2 — Dados

ID	Tarefa	Entrega	Aceitação	Est.
F04	Verificar FIRMS (spike 2h)	`docs/FIRMS_ACCESS.md` preenchido: sim/não, endpoint, erro, produto/versão/data	Timebox respeitado, bloqueio documentado	2h

ID	Tarefa	Entrega	Aceitação	Est.
F05	Amostras	`modis_sample.csv` + `viirs_sample.csv` (máx 500 linhas) + `detections_mock.json` 20 linhas	M3/M4 correm sem FIRMS	1.5h
F06	Pipeline normalização	`normalize.py`: valida lat/lon/data, preserva `raw_properties`, relatório lidas/válidas/rejeitadas+motivos	`pytest -q` passa	3h
F07	Metodologia v1	`docs/methodology.md`: regra grelha 0.05° + mesmo dia, limitações	Explicável em 2 min	1.5h

Passos M2 em detalhe:

1. Pedir MAP_KEY em https://firms.modaps.eosdis.nasa.gov/api/map_key/

2. Testar 1 chamada área Angola + 1 download em https://firms.modaps.eosdis.nasa.gov/download/

3. Inspeccionar 50 linhas de cada sensor: colunas, data/hora, confidence, FRP.

4. Implementar validação: lat [-90,90], lon [-180,180], data parseável, campos opcionais não crasham.

5. Nunca apagar rejeitados sem contar motivo.

6.3 M3 — Backend + DB

ID	Tarefa	Entrega	Aceitação	Est.
F08	Modelo BD	`database/schema.sql`: 4 tabelas + índices GIST(geom), BTREE(acquired_at,sensor), unique matches	Aplica sem erro ou adiamento escrito	2h
F09	API inicial	`GET /api/health`, `GET /api/detections?bbox,startDate,endDate,sensor,limit`, `GET /api/meta` com mock	Frontend sem ler CSV directo	3h

Contrato F09:

• `bbox=minLon,minLat,maxLon,maxLat` obrigatório para mapa.

• `limit` default 100, máx 1000. `bbox` inválido -> 400 `{error,code,details}`.

• `GET /api/meta`: `{periodo, produtos, last_import, is_sample}`.

6.4 M4 — Frontend

ID	Tarefa	Entrega	Aceitação	Est.
F10	Layout base	Next.js: header, mapa placeholder, legenda, banner TESTE/REAL, estados loading/vazio/erro	`npm run dev` abre sem crash	2h

ID	Tarefa	Entrega	Aceitação	Est.
F11	Mapa inicial	Leaflet + cluster, cores por sensor, popup, filtros data/sensor com mock	200 pontos filtráveis	3h

Regras M4:

• Nunca renderizar >1000 pontos sem cluster/bbox.

• Legenda sempre = cores reais.

• Série temporal distingue `0` (zero obs.) vs `null` (sem dados).

6.5 M5 — UX/Testes/Docs

ID	Tarefa	Entrega	Aceitação	Est.
UX1	Guia visual	Paleta por sensor, layout, banner proveniência	Legível sem saber sensoriamento	1h
T1	Checklist manual	Testar abrir, mapa, filtro vazio, API parada	Registado em F12	1h
D1	Apoio demo	Slides metodologia + limitações + fonte	Sem afirmação excessiva	1h

Critérios de conclusão Sprint 1: repo organizado, FIRMS verificado ou bloqueio documentado, amostra disponível, pipeline valida, metodologia escrita, API mock, mapa mock, demo offline possível.

7. Sprint 2 — Integração do fluxo (pós 23 Out)

Objectivo: trocar mock por real onde houver dados, ligar ponta-a-ponta.

• M1: congela contrato OpenAPI, gere cortes. Se atraso, corta RF12/RF13.

• M2: melhora validação (duplicados, `confidence_raw` vs `norm`), gera `matches.csv` experimental com `rule_version`, testes `small_valid/invalid.csv`.

• M3: liga Postgres/SQLite, paginação cursor, `GET /analytics/timeseries`, `GET /analytics/sensors`, testes Supertest.

• M4: liga à API real, Recharts por sensor, indicadores totais/por sensor/válidos/rejeitados, `is_sample` visível.

• M5: teste ponta-a-ponta `CSV -> pipeline -> API -> mapa = N pontos`, regista em `known_issues.md`.

Entrega Sprint 2: `sample.csv real -> mapa filtrável` funciona.

8. Sprint 3 — Validação e apresentação

Objectivo: demo estável e honesta.

• M1: guião 7 passos (problema->abordagem->demo->harmonização->transparência->impacto->próximos), vídeo 90s, screenshots backup, README final com citação NASA.

• M2: congela datasets demo + relatório importação para slides, teste sensibilidade limiares (500m vs 1km vs 2km).

• M3: cap performance, documentos endpoints, `npm test` + `build` verdes.

• M4: polir mobile, lendas, performance cluster, modo offline com `data/samples/`.

• M5: ensaio geral, cronómetro, plano B sem internet.

Mensagem central: tornar observações comparáveis preservando origem e limitações.

9. Backlog completo e prioridades

ID	Requisito	Prioridade	Dono	Sprint
RF01	Importar amostra MODIS+VIIRS	P0	M2	1
RF02	Validar coords e datas	P0	M2	1
RF03	Normalizar campos	P0	M2	1
RF04	Preservar proveniência	P0	M2/M3	1
RF05	Mapa observações	P0	M4	1
RF06	Distinguir fontes	P1	M4	1-2
RF07	Filtrar por período	P1	M4/M3	1-2
RF08	Série temporal	P1	M4/M3	2
RF09	Comparar por sensor	P1	M2/M4	2
RF10	Registar imports/erros	P1	M2/M3	1-2
RF11	Detalhe observação	P2	M3/M4	2-3
RF12	Agregar por região	P2	M2/M3	3*
RF13	Tendências/anomalias	P2	M2	3*
RF14	Docs metodologia	P0	M2/M1	1

*P2 só se P0+P1 verdes dia 21.

10. Git e colaboração — resumo operacional

1. `git pull origin main` -> `git checkout -b feat/nome` -> muda pouco -> testa -> `commit` -> `push` -> PR -> 1 approval -> merge -> delete branch.

2. Branches: `feat/*`, `fix/*`, `docs/*`, `test/*`, `chore/*`.

3. Commits: `feat:`, `fix:`, `docs:`, `test:`, `chore:`.

4. PR pequeno <200 linhas, com Como testar + mock/real + screenshots se UI.

5. Proteger `main`: exige PR + 1 approval. Sem excepções.

11. Testes por dono

• M2 `pytest`: lat/lon inválidos rejeitados, data má rejeitada, opcional ausente não crasha, contagem rejeitados correcta, proveniência preservada.

• M3 Vitest/Supertest: health 200, bbox mau 400, limit cap, paginação, erro JSON consistente.

• M4 manual: abre, mapa aparece, inválidas não partem, legenda correcta, vazio/erro compreensíveis.

• M5 integração: fluxo ponta-a-ponta com fixtures `small_valid.csv` / `small_invalid.csv`.

• CI sugerido: job Python `pytest` + job Node `npm ci && npm test` + `npm run build` frontend.

12. Riscos e cortes

Risco	Sinal	Corte M1
FIRMS bloqueada	F04 = não dia 18	Ficar em mock + download manual, API pendente
Sem 2 sensores	Só 1 CSV dia 19	Normalização separada, sem alegar harmonização validada
DB atrasa	Sem schema dia 20	JSON/SQLite, schema documentado
Mapa atrasa	Sem cluster dia 22	Página mínima + screenshot estático identificado
Scope creep	Auth/alertas/ML pedidos	Adiar sem discutir, foco dados+demo

Regra de ouro dia 21: se `CSV real -> mapa filtrável` não passa, cortam-se matches, RF12, RF13.

13. Checklists finais

Demo gate (F12): builds verdes, offline com samples testado, banner TESTE/REAL visível, filtros actualizam mapa+gráfico, vazio mostra mensagem, erro API mostra mensagem, metodologia 1 slide, vídeo+screenshots backup, linguagem `detecções` não `incêndios`, `known_issues.md` actualizado.

Entrega: README com como correr, fonte/produto/versão/período citados, metodologia, limitações, próximos passos, sem segredos.

Fim. Documento vivo: M1 actualiza após cada Sprint Review. Decisões novas vão para `docs/decisions.md`, não aqui.